PardusGuard

System Hardening & Security Assessment Tool

Detailed Project Plan | TEKNOFEST 2026 | Development Category

1. PROJECT OVERVIEW

PardusGuard is an open-source system hardening and security assessment tool built specifically for the Pardus Linux operating system. It bridges the gap between powerful but complex security tools (like Lynis, OpenSCAP) and everyday users who lack the expertise to interpret their output or act on their recommendations.

The tool performs automated security analysis across six domains, scores the system's overall risk posture, presents findings in a GTK4 graphical dashboard, and lets the user apply recommended fixes either automatically or step-by-step — all without needing terminal knowledge.

1.1 Problem Statement

Pardus is widely adopted by Turkish government agencies, universities, and schools. Most of these users are not security specialists. Currently:

- No official Pardus tool exists to assess and harden a fresh or running installation
- Command-line tools such as ufw, systemctl, ss, and find require significant Linux expertise
- Security misconfigurations — open ports, weak SSH, SUID binaries, world-writable files — persist silently on many systems
- Security awareness among non-expert users is critically low

1.2 Proposed Solution

PardusGuard provides a single unified interface that:

- Scans the system automatically across all major hardening domains
- Assigns a numeric risk score (0–100) with colour-coded severity levels
- Explains each finding in plain language with a concrete fix
- Allows one-click automatic remediation for safe, well-understood issues
- Exports full security reports in PDF and JSON formats
- Logs every action taken for auditability

2. GOALS & SUCCESS CRITERIA

Goal	Measurable Success Criterion	Priority
Automated security scanning	All 6 scan modules run without error on Pardus 25.0	**Critical**

Goal	Measurable Success Criterion	Priority
Risk scoring engine	Score produced in < 5 seconds on standard hardware	Critical
GTK4 graphical dashboard	Full UI functional, no crashes in 30-minute session	Critical
One-click hardening	At least 10 safe auto-fix actions implemented	High
Report export (PDF/JSON)	Export produces valid, readable file every time	High
PolicyKit privilege handling	No sudo prompts in terminal; all auth via pkexec dialog	High
Debian package (.deb)	Installs and uninstalls cleanly on Pardus 25.0	High
Community contribution ready	Public GPL-3.0 repo with CONTRIBUTING.md and CI/CD	Medium

3. SYSTEM ARCHITECTURE

PardusGuard follows a layered architecture with a strict separation between the scanning engine, the data/scoring layer, and the presentation layer. This makes each part independently testable and allows the tool to be used headlessly (CLI) as well as through the GUI.

3.1 Architectural Layers

Layer	Components	Responsibility
Presentation	GTK4 dashboard, report renderer, notification manager	All user interaction; reads from data layer only
Orchestration	scan_runner.py, scheduler, event bus	Coordinates scans, passes results to scoring, triggers UI updates
Scan Engines	6 specialist scanner modules (see §3.2)	Runs native Linux tool calls; returns structured findings
Scoring & Analysis	risk_scorer.py, finding_classifier.py	Converts raw findings into scored, categorised results
Remediation	fix_engine.py, action_registry.py	Executes pre-vetted fix actions via PolicyKit; logs everything
Data / Storage	SQLite3 database, JSON config, log files	Persists scan history, user settings, fix audit trail
Privilege Bridge	PolicyKit (.policy file) + pkexec wrapper	Secure privilege escalation without running GUI as root

3.2 The Six Scan Engines

Each engine is an independent Python module that calls native Linux tools, parses their output, and returns a standardised list of Finding objects.

Engine 1 — Port & Network Scanner

- Calls: ss -tulnp, nmap (local)

Engine 2 — SSH Configuration Auditor

- Reads: /etc/ssh/sshd_config

- Detects open TCP/UDP ports
- Identifies listening services by PID
- Flags dangerous ports (23, 21, 3389, 5900)
- Checks for unexpected external exposure
- Suggests nftables rules to close ports

- Checks: PermitRootLogin, PasswordAuth
- Validates: Protocol version, ciphers
- Checks: MaxAuthTries, LoginGraceTime
- Detects: AllowUsers / AllowGroups absence
- Applies: sshd_config patches via fix engine

Engine 3 — Service Auditor

- Calls: systemctl list-units --all
- Lists all enabled/running services
- Cross-references known unnecessary services
- Flags services with root execution context
- Suggests: systemctl disable <service>
- Checks for insecure legacy services (telnet, rsh)

Engine 4 — Kernel & OS Hardening

- Reads: /proc/sys/kernel/* sysctl values
- Checks: ASLR, SYN cookies, IP forwarding
- Validates: kernel.randomize_va_space = 2
- Checks: core dump restrictions
- Verifies: /etc/sysctl.conf hardening params
- Applies: sysctl -w <param>=<value> fixes

Engine 5 — File Permission Auditor

- Calls: find with -perm flags
- Detects world-writable files & directories
- Finds SUID/SGID binaries not in whitelist
- Checks /etc, /tmp, /var permissions
- Verifies critical file ownership (passwd, shadow)
- Suggests chmod / chown remediation commands

Engine 6 — User & Authentication Auditor

- Reads: /etc/passwd, /etc/shadow, /etc/sudoers
- Detects: empty password accounts
- Finds: UID 0 accounts other than root
- Checks: sudo NOPASSWD entries
- Validates: password aging policies
- Checks: PAM configuration for weak auth

4. FEATURE SPECIFICATION

4.1 Risk Scoring Engine

Every Finding produced by a scan engine is assigned a severity (Critical / High / Medium / Low / Info) and a weight. The overall system score is calculated as:

```
Score = 100 - SUM(finding.weight * severity_multiplier)

Severity multipliers:  Critical=10  |  High=5  |  Medium=2  |  Low=0.5  |  Info=0
```

Score Range	Label	Dashboard Colour	Recommended Action
85 – 100	Secure	**Green**	Review Info findings only

Score Range	Label	Dashboard Colour	Recommended Action
65 – 84	Moderate Risk	**Yellow**	Address High findings this week
40 – 64	High Risk	**Orange**	Apply auto-fixes immediately
0 – 39	Critical	**Red**	System requires urgent hardening

4.2 GTK4 Dashboard — Page Breakdown

Page / View	Key Elements	Data Source
Home / Overview	Overall score dial, severity breakdown bar, last-scan timestamp, Quick Scan button	risk_scorer.py latest result
Scan Results	Findings list grouped by engine, expandable rows with description + fix preview, severity badges	All 6 scan engines
Hardening Actions	Actionable fix cards, Apply / Skip / Explain buttons, progress indicator, before/after preview	fix_engine.py action_registry
Network View	Live open ports table, protocol/PID columns, Block Port button, nftables rule preview	Port Scanner Engine
Reports	Export to PDF button, Export to JSON button, scan history timeline, comparison between scans	SQLite3 history + report_renderer.py
Settings	Scan schedule, exclusion lists, notification preferences, theme toggle, about page	config_manager.py
Logs	Timestamped audit log of all scans and fixes applied, filter by type/date, export	log_manager.py

4.3 Remediation Engine — Fix Categories

The fix engine applies only pre-vetted, reversible actions. Every action is logged before and after execution. Users can preview the exact command before applying it.

Fix Type	Examples	Auto-Apply Safe?
SSH config hardening	Set PermitRootLogin no, MaxAuthTries 3, disable password auth	**Yes — with backup**
Service disable	systemctl disable telnet, rsh, avahi-daemon	**Yes**
nftables rule addition	Block port 23, 21, 5900 with nft add rule	**Yes — rule preview shown**
sysctl hardening	Enable ASLR, SYN cookies, disable IP forwarding	**Yes — /etc/sysctl.conf updated**
File permission fix	chmod 644 on world-writable /etc files	**Yes — with confirmation**
SUID binary removal	chmod u-s on non-whitelisted SUID binaries	**Manual only — user must confirm**
Password policy	Set /etc/login.defs PASS_MAX_DAYS, PASS_MIN_DAYS	**Yes**
sudoers NOPASSWD removal	Comment out NOPASSWD entries in /etc/sudoers	**Manual only — high risk**

5. PROJECT STRUCTURE

```
pardusguard/
├── src/
│   ├── main.py                    # Entry point
│   ├── main_window.py             # GTK4 application window
│   ├── scan_runner.py             # Orchestrates all 6 engines
│   ├── risk_scorer.py             # Scoring & classification
│   ├── fix_engine.py              # Remediation executor
│   ├── action_registry.py         # Registry of all fix actions
│   ├── report_renderer.py         # PDF/JSON export
│   ├── log_manager.py             # Audit logging
│   ├── config_manager.py          # Settings persistence
│   ├── engines/
│   │   ├── port_scanner.py
│   │   ├── ssh_auditor.py
│   │   ├── service_auditor.py
│   │   ├── kernel_hardening.py
│   │   ├── file_permission.py
│   │   └── user_auditor.py
│   └── ui/
│       ├── dashboard.py           # Overview page
│       ├── scan_results.py        # Findings list page
│       ├── hardening_page.py      # Fix cards page
│       ├── network_view.py        # Port table page
│       ├── reports_page.py
│       ├── settings_page.py
│       └── logs_page.py
├── data/
│   ├── org.pardus.pardusguard.policy   # PolicyKit
│   ├── pardusguard.desktop
│   └── risk_weights.json               # Tunable scoring weights
├── tests/
│   ├── test_port_scanner.py
│   ├── test_ssh_auditor.py
│   ├── test_risk_scorer.py
│   ├── test_fix_engine.py
│   └── test_report_renderer.py
├── debian/                        # Debian packaging
├── docs/
│   ├── README.md
│   ├── USER_GUIDE.md
│   ├── TECHNICAL.md
│   └── CONTRIBUTING.md
├── setup.py
├── requirements.txt
└── LICENSE (GPL-3.0)
```

6. TECHNOLOGY STACK

Component	Technology	Version	Justification
Language	Python	3.10+	Native Pardus support; rich ecosystem for system scripting
GUI Framework	GTK4 + PyGObject	4.x	GTK4 is the current Pardus/GNOME standard; modern, accessible
Port/Network Scan	ss (iproute2) + nmap	System	ss is always available; nmap for deeper local scanning
Firewall Integration	nftables (nft CLI)	System	Modern successor to iptables; default on Pardus 23+
Service Management	systemctl (systemd)	System	Standard Pardus service manager

Component	Technology	Version	Justification
Privilege Escalation	PolicyKit + pkexec	0.105+	Secure, auditable; no sudo prompts in terminal
Database	SQLite3	3.x	Zero-dependency; scan history and settings storage
Report Export	ReportLab (PDF) + json	4.x	PDF generation without external process dependency
Testing	pytest + pytest-cov	7.x	Industry standard; coverage reporting
Packaging	debhelper + dh-python	11+	Native Pardus/Debian packaging
CI/CD	GitHub Actions	N/A	Automated test runs on every commit; required for open-source
License	GPL-3.0	N/A	Required for open-source competition submission

7. 20-WEEK DEVELOPMENT PLAN

The plan is divided into six phases aligned with the TEKNOFEST competition timeline. Each phase ends with a defined deliverable that can be demonstrated independently.

Phase 1 — Foundation & Architecture (Weeks 1–2)

Goal: Working development environment, agreed architecture, all tooling in place before a single feature line is written.

Week	Task	Deliverable	Owner Area
Week 1	Set up Pardus 25.0 VM; install all dependencies; create public GitHub repo with CI/CD skeleton; agree on coding standards (PEP 8, type hints, docstrings)	GitHub repo live; CI runs on push; dev environment documented	All
Week 1	Study native tool outputs: ss, systemctl, find, sysctl, /etc/ssh/sshd_config — document exact parsing targets for each engine	Parsing spec document in /docs	Scan Engine Lead
Week 2	Design Finding dataclass schema; design database schema (SQLite3); design PolicyKit policy file; write risk_weights.json first draft	Architecture document; Finding schema; DB schema	Backend Lead
Week 2	Design GTK4 wireframes for all 7 pages; define colour palette and severity badge system; reference existing Pardus apps for HIG compliance	Wireframes in /docs/screenshots/	UI Lead

Phase 1 Exit Criteria

- Public GitHub repository is live and accessible
- GitHub Actions CI runs successfully on the skeleton project
- All team members have working Pardus 25.0 dev environment
- Architecture document and wireframes reviewed and approved by team

Phase 2 — Scan Engines (Weeks 3–9)

Goal: All six scan engines producing accurate, structured Finding objects. Each engine is independently testable with unit tests before UI is connected.

Week	Engine / Module	Key Implementation Tasks	Test Target
Week 3	Engine 1: Port Scanner	Wrap ss -tulnp; parse output into structured rows; identify PIDs; flag RISKY_PORTS list; return Finding objects with severity	test_port_scanner.py — 15 test cases
Week 4	Engine 2: SSH Auditor	Parse /etc/ssh/sshd_config key-value pairs; check 12 security parameters; each misconfigured param = 1 Finding with suggested fix value	test_ssh_auditor.py — covers all 12 params
Week 5	Engine 3: Service Auditor	Query systemctl list-units --all --output=json; filter enabled services; cross-reference against UNNECESSARY_SERVICES list; flag root-context services	test_service_auditor.py — mock systemctl output
Week 6	Engine 4: Kernel Hardening	Read /proc/sys/* and sysctl -a output; check 15 key kernel params; compare against hardened baseline values in risk_weights.json	test_kernel_hardening.py — uses /proc fixtures
Week 7	Engine 5: File Permissions	Run find commands for SUID/SGID, world-writable, critical file perms; use whitelist for known-safe SUID binaries; check /etc ownership	test_file_permission.py — temp filesystem fixtures
Week 8	Engine 6: User Auditor	Parse /etc/passwd, /etc/shadow (via pkexec), /etc/sudoers; detect UID 0 duplicates, empty passwords, NOPASSWD sudo, missing password aging	test_user_auditor.py — mock /etc files
Week 9	Orchestration + Scoring	Build scan_runner.py to run all 6 engines sequentially/parallel; build risk_scorer.py to aggregate findings into final score; full integration test	Full integration test: scan → score → structured output

Phase 2 Exit Criteria

- All 6 engines return correct Finding objects on Pardus 25.0
- risk_scorer.py produces a correct numeric score from combined findings
- Unit test suite passes with ≥ 70% coverage on all engine modules
- No engine crashes or hangs on a clean Pardus installation

Phase 3 — GTK4 User Interface (Weeks 10–14)

Goal: Fully functional 7-page GTK4 application connected to live backend data. All navigation working. Pardus HIG compliant.

Week	UI Component	Key Tasks
Week 10	Application Shell + Home Dashboard	Build GTK4 application window with sidebar navigation; implement Overview page with score dial (Cairo drawing), severity bar, last-scan info, Quick Scan button with async threading via GLib.idle_add
Week 11	Scan Results Page	Implement findings list grouped by engine using GTK4 ListBox; expandable rows showing description, severity badge, and fix preview; filter toolbar by severity and engine
Week 12	Hardening Actions Page + Network View	Build fix cards with Apply / Skip / Explain actions; progress indicator during fix application; implement Network View with live port table and Block Port capability

Week	UI Component	Key Tasks
Week 13	Reports Page + Logs Page	Scan history timeline; PDF/JSON export buttons wired to report_renderer.py; Logs page with timestamped audit entries, type filter, date filter, and export
Week 14	Settings Page + Polish Pass	All settings wired to config_manager.py; notification preferences; scan scheduling UI; full accessibility pass (keyboard navigation, screen reader labels); dark/light theme support

Phase 3 Exit Criteria

- All 7 pages are navigable with no crashes in a 30-minute session
- Dashboard displays live score from a real scan of the Pardus VM
- Findings list accurately reflects all engine outputs
- No UI thread blocking — all scans run in background threads

Phase 4 — Remediation Engine & Reports (Weeks 15–16)

Goal: Safe, audited, PolicyKit-authenticated fix application. PDF and JSON report export working.

Week	Module	Key Tasks
Week 15	fix_engine.py + action_registry.py	Implement action registry with all pre-vetted fix actions (SSH config patch, service disable, sysctl write, nftables rule, file chmod, password policy); each action: validate → backup → apply → verify → log; wire PolicyKit pkexec for privileged actions; implement undo/rollback for each action type
Week 16	report_renderer.py + log_manager.py	PDF report: cover page, score summary, findings table, recommended actions, scan metadata; JSON report: full machine-readable Finding list with all fields; log_manager: append-only audit log with action type, timestamp, before/after state, user confirmation status

Phase 4 Exit Criteria

- All 8 auto-fix types apply correctly and are logged
- Each fix can be previewed before application
- PolicyKit authentication dialog appears for privileged fixes (no terminal sudo)
- PDF export produces a readable, properly formatted report
- JSON export passes schema validation

Phase 5 — Testing & Optimisation (Weeks 17–18)

Week	Focus	Key Tasks
Week 17	Comprehensive Testing	Complete all unit tests to reach ≥ 85% coverage; write integration tests for full scan → score → fix → report pipeline; manual test all UI flows on Pardus 25.0 VM; test edge cases: no network, no SSH service, minimal Pardus install, root-only files
Week 18	Bug Fixes + Performance	Fix all bugs found in Week 17; profile scan speed — target < 10 seconds for full scan; optimise file permission engine (most CPU-intensive); ensure all background threads properly cleaned up; memory usage review

Phase 5 Exit Criteria

- Zero critical bugs; zero high-severity bugs
- Test suite passes at ≥ 85% coverage
- Full scan completes in under 10 seconds on standard hardware
- All 7 UI pages tested manually with no crashes

Phase 6 — Packaging, Documentation & Submission (Weeks 19–20)

Week	Focus	Key Tasks
Week 19	Debian Packaging + Final Docs	Write debian/control, debian/rules, debian/postinst (ensure nftables and systemd available); write README.md, USER_GUIDE.md, TECHNICAL.md, CONTRIBUTING.md; take screenshots of all 7 pages; record 3-minute demo video
Week 20	Competition Submission	Build final .deb package; test install/uninstall on clean Pardus 25.0 VM; submit pull request to github.com/pardus; submit talep on talep.pardus.org.tr with repo link, description, screenshots; verify all competition checklist items

Phase 6 Exit Criteria — Final Submission Checklist

- .deb package installs and uninstalls cleanly on Pardus 25.0
- All documentation written: README, USER_GUIDE, TECHNICAL, CONTRIBUTING
- Screenshots of all 7 pages included in /docs/screenshots/
- Demo video recorded and linked in README
- GPL-3.0 LICENSE file present in root
- Pull request submitted to github.com/pardus
- Talep submitted on talep.pardus.org.tr before 31 July 2026

8. MILESTONE TIMELINE

Milestone	Target Date	Deliverable
M1 — Foundation Complete	7 Mar 2026	GitHub repo live, CI/CD active, architecture docs, wireframes
M2 — All 6 Engines Working	25 Apr 2026	scan_runner produces scored output; 70% test coverage
M3 — Full UI Working	6 Jun 2026	All 7 GTK4 pages navigable; connected to live scan data
M4 — Hardening + Reports	20 Jun 2026	All fix types working; PDF/JSON export complete
M5 — Testing Complete	4 Jul 2026	Zero critical bugs; ≥85% coverage; full scan <10 sec
M6 — Competition Submission	**31 Jul 2026**	**.deb packaged; talep submitted; GitHub PR open**
Results Announced	28 Aug 2026	Winner announcement
TEKNOFEST Şanlıurfa	30 Sep – 4 Oct 2026	Award ceremony

9. COMPETITION ALIGNMENT & SCORING STRATEGY

The jury evaluates on: type and severity of findings addressed, completeness of description, quality of solution, and code quality. Here is how PardusGuard maximises each criterion:

Jury Criterion	PardusGuard Response	Score Impact
Security Vulnerabilities (Critical category)	All 6 engines directly address security zafiyetleri — the highest-weighted finding type in the scoring rubric	**Maximum**
Functional correctness	Unit + integration tests; CI/CD green on every commit; tested on Pardus 25.0	**Maximum**
Code quality	PEP 8 enforced by flake8 in CI; type hints; full docstrings; modular architecture	**High**
Usability improvement	Non-expert users can harden their system in < 5 minutes with no terminal knowledge	**High**
Documentation quality	README, USER_GUIDE, TECHNICAL, CONTRIBUTING — all written by Week 19	**High**
Open-source contribution readiness	CONTRIBUTING.md, issue templates, PR template, GitHub Actions CI from Week 1	**Medium**
Pardus ecosystem compatibility	GTK4 HIG compliant; .deb packaged; PolicyKit integrated; GPL-3.0	**High**

10. RISK REGISTER

Risk	Likelihood	Impact	Mitigation
GTK4 not fully stable on Pardus 25.0	Medium	High	Test GTK4 vs GTK3 compatibility in Week 1; fall back to GTK3 if GTK4 has blockers — architecture is identical
/etc/shadow requires root; pkexec adds complexity	Medium	High	Design PolicyKit policy and pkexec wrapper in Week 2 before any engine uses it; test on clean VM
nftables syntax differs from iptables — wrong rules possible	Low	Critical	All nftables rules are pre-validated in action_registry; preview shown to user before apply; rollback implemented
File permission engine is slow on large filesystems	High	Medium	Use find with -maxdepth limits; run in background thread; add progress indicator; results cached per session
Competition submission deadline missed	Low	Critical	Milestone M6 is set to 31 Jul; all docs written in Week 19; team lead monitors talep.pardus.org.tr deadlines weekly
Engine produces false positives (normal binaries flagged)	Medium	Medium	Maintain whitelist of known-safe SUID binaries; tune risk_weights.json; user can add exclusions in Settings

11. WHY PARDUSGUARD WINS

Competitive Advantages | **Comparison vs Existing Tools**

- No equivalent tool exists in the Pardus ecosystem
- Addresses all 5 competition finding types: functional, performance, usability, security, localisation
- Security zafiyetleri findings = highest jury scoring weight
- Real-world value for government and education Pardus users
- Clean GTK4 UI shows polish that impresses non-technical judges
- Fully automated CI/CD signals serious engineering practice
- GPL-3.0 + CONTRIBUTING.md = community longevity argument
- One-click remediation is a feature no existing Pardus tool has

Lynis
CLI only, generic Linux, no Pardus integration, overwhelming output for non-experts

OpenSCAP
Extremely complex, XML-based, government compliance focus, no GUI

ufw / nftables
Single-purpose, no scanning, no reporting, terminal only

ClamAV
Antivirus only, does not address system hardening misconfigurations

PardusGuard
GTK4 GUI, Pardus-specific, all-in-one, non-expert friendly, open-source

Scan. Score. Harden. Educate.

PardusGuard — Making Linux Security Accessible to Everyone